系统设计速成 · 替代 Alex Xu，一晚上能看完

给 11–12 点 Kyle 那一小时（写代码 + 系统设计）。每个概念：先比方，再要点，再一个数字，最后一句英文。黄底「你的项目里」= 答题时拿来当例子的东西。不重复代码对照页和补充手册已有的内容。

0 · 系统设计题怎么答（4 步，30 分钟分法）

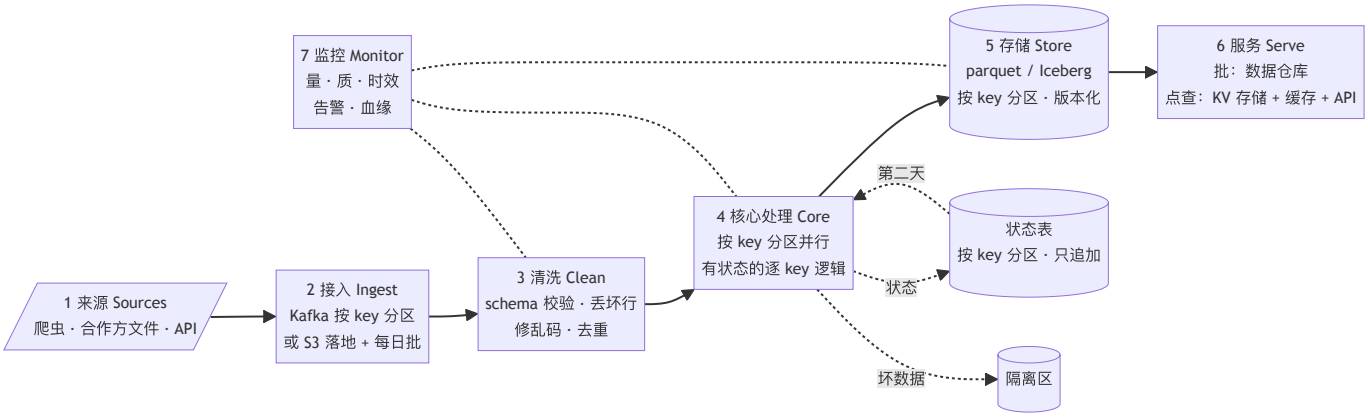
- **第 1 步 · 问清楚要求（3–5 分钟）（clarify requirements）**：功能要什么、不要什么；规模多大（用户数、每天多少数据、读多写多）；延迟要求（秒级还是天级）；一致性要求（能不能晚一点看到）。先问再画，别一上来就画框。
- **第 2 步 · 估算（3 分钟）（back-of-envelope）**：QPS、存储、带宽，算到数量级就行（见第 1 节）。
- **第 3 步 · 画高层图（10 分钟）（high-level design）**：数据从哪来 → 经过什么 → 存哪 → 谁来读。每个框一句话说为什么要它。
- **第 4 步 · 深挖 1–2 个点（10 分钟）（deep dive）**：他最关心的那个瓶颈。说清取舍：选了 A 放弃了 B，因为 C。
- 最后 2 分钟：瓶颈在哪、怎么扩、怎么监控、下一步。

Before I draw anything, can I check a few things: what's the daily volume, how fresh does the result need to be, and who reads it? Then I'll estimate the numbers, sketch the pipeline, and go deep on the part you care most about.

开口模板。他们是数据公司，题目八成是「数据流水线」而不是「设计 Twitter」。

万能模板图（答不出来就画这个，再往里填）

任何「数据流水线」题都是这六个框。先把框画出来，再一个一个说每个框里用什么、为什么。



照着说的顺序（每框一句）：

- 1 来源：数据从哪来、每天多少、格式是什么、会不会迟到。
- 2 接入：要秒级就 Kafka（按 user_id 分区保序），要天级就 S3 落文件每天跑一批。
- 3 清洗：schema 契约、丢坏行不丢整批、修乱码、按内容哈希去重。
- 4 核心：按 key 分区让同一个人的数据在一台机器上；逐 key 的有状态逻辑（这就是你的状态机）；一个 key 出错隔离不阻塞。
- 5 存储：列式 parquet；表格式给事务、分区裁剪、时间旅行；状态按 key 分区只追加。
- 6 服务：分析查询走仓库；点查走 KV + 缓存 + API。
- 7 监控：每次运行出指标，突变告警，必须为 0 的硬断言。
- 最后说三句：瓶颈在 4（算力线性加机器）和 5（小文件）；扩容靠分区数；取舍是 X 换 Y 因为 Z。

Any data pipeline is the same six boxes: sources, ingest, clean, core processing partitioned by key, storage, and serving, with monitoring across all of them and a state store feeding the next run. Let me draw that and then fill in each box for this problem.

1・估算（back-of-envelope）

比方：装修前先量房间，不用精确到毫米，知道是 50 平还是 500 平就够。

记住这几个数	用法
1 天 ≈ 86,400 秒 ≈ **10 万秒**	每天 1 亿请求 ÷ 10 万秒 ≈ 1,000 QPS
1 KB × 10 亿 = 1 TB	每行 1 KB、10 亿行 = 1 TB
一台机器：内存 64–256 GB，SSD 几 TB，网卡 1–10 Gbps	超过就要分机器
内存读 100 ns、SSD 读 100 μs、跨机房网络 100 ms	每差 1000 倍；能放内存就别读盘
一个 CPU 核每秒能处理多少行：看每行干多少活（见下）	你实测 3.5 万行/核秒（含 Python 模糊匹配）

「一个核每秒处理几万到几十万行」怎么来的：一个 CPU 核每秒大约做 10^9 次简单操作。处理一行要做多少次操作，决定每秒能过多少行：

每行干的活	每行大约几次操作	每核每秒多少行	例子
纯向量化：过滤、加列、类型转换（Polars/Spark 的列式计算）	10^2–10^3	10^6–10^7（百万到千万）	你的 conform、clean、分桶
一般逐行逻辑：解析、几次比较、写出	10^4–10^5	10^4–10^5（几万到几十万）	普通 ETL 的估算默认值
重逻辑：Python 循环 + 字符串相似度 + 多候选打分	10^4–10^5，但 Python 本身慢 10–100 倍	10^4 左右	你的 step + match：3.5 × 10^4 行/核秒（指纹跳过一半帮了大忙）
每行调一次模型 / LLM	10^8 以上，或网络往返	10–100	这就是为什么不能 LLM 逐对判

用法：估算时先判断「每行是哪一档活」，再乘以行数得核秒。被问「怎么知道 3.5 万」：实测，1150 万行 ÷ (78 秒 × 4 核) ≈ 3.7 万，取 3.5 万保守。

A core does about a billion simple operations a second. Vectorised column work is a few hundred operations per row, so millions of rows per second; row-by-row logic in Python with fuzzy matching is tens of thousands per second, which is what I measured, 35 thousand rows per core-second. Calling a model per row drops that to tens, which is why an LLM can't sit on the hot path.

你的项目里：1000 亿行 × 约 500 字节 ≈ 50 TB 输入；3.5 万行/核秒 → 1000 亿 ÷ 3.5 万 ≈ 286 万核秒 ≈ 800 核时；64 核约 12 小时。输出约 1.08 倍输入。

At roughly 500 bytes a row, 100 billion rows is about 50 terabytes. At 35 thousand rows per core-second that's about 800 core-hours, so 12 hours on 64 cores or a bit over an hour on 640.

数量级换算表（10 的几次方）

10 的几次方	叫法	字节单位	直观
10^3	千 thousand（K）	1 KB ≈ 一条 1000 字的记录	一个人一段经历的所有字段约 0.5 KB
10^6	百万 million（M）	1 MB ≈ 一张照片	一百万行 × 1 KB = 1 GB
10^9	十亿 billion（G / B）	1 GB ≈ 一部电影	十亿行 × 1 KB = 1 TB
10^{12}	万亿 trillion（T）	1 TB ≈ 一块硬盘	1000 亿行 × 0.5 KB = 50 TB
10^{15}	千万亿 quadrillion（P）	1 PB = 1000 块硬盘	大公司数据湖级别

中文和英文的坑：中文按万进位，英文按千进位。

中文	数字	英文
1 万	10^4	10 thousand
10 万	10^5	100 thousand
100 万	10^6	1 million
1000 万	10^7	10 million
1 亿	10^8	100 million
10 亿	10^9	1 billion
1000 亿	10^{11}	100 billion
1 万亿	10^{12}	1 trillion

时间：1 天 = 86,400 秒 ≈ 10^5（算 QPS 时直接除 10 万）；1 月 ≈ 2.6×10^6 秒；1 年 ≈ 3.15×10^7 秒。

延迟（相差 1000 倍一档）：内存 100 ns = 10^{-7} 秒 → SSD 100 μs = 10^{-4} 秒 → 跨机房网络 100 ms = 10^{-1} 秒。

你的题目用这张表算一遍：

- 输入 11.5M 行 = 1.15×10^7；150k 人 = 1.5×10^5；每人约 77 行。
- 1000 亿行 = 10^{11}；× 500 B（5×10^2）= 5×10^{13} B = 50 TB。
- 3.5 万行/核秒 = 3.5×10^4；$10^{11} \div 3.5 \times 10^4 \approx 2.9 \times 10^6$ 核秒；÷ 3600 ≈ 800 核时。
- 64 核：800 ÷ 64 = 12.5 小时；640 核：1.25 小时。
- 每天 3 亿新行 = 3×10^8；$\div 3.5 \times 10^4 \approx 8.6 \times 10^3$ 核秒 ≈ 2.4 核时；4 核约 36 分钟。
- 1 万 QPS 的点查：$\times 10^5$ 秒/天 = 10^9 次/天。

I round everything to powers of ten: a day is ten to the fifth seconds, a kilobyte times a billion rows is a terabyte, and memory, SSD and network are each a thousand times apart. 100 billion rows at 500 bytes is 50 terabytes; at 35 thousand rows per core–second that's about 800 core–hours.

2 · 分区 / 分片（partitioning / sharding） + 一致性哈希

比方：一个图书馆放不下，按书名首字母分到 26 个馆。分区就是「按什么键把数据切开放到不同机器」。

- **按键取模（modulo / hash partitioning）**（你的 user_id % 64）：简单、均匀。缺点：桶数一变，几乎所有数据都要搬家（100 % 64 = 36，100 % 65 = 35）。
- **按范围（range partitioning）**：按数值区间切，user_id 1 到 100 万放机器 1，100 万到 200 万放机器 2，以此类推（像图书馆按编号分楼层）。好处：范围查询只问一台。坏处：容易热点，新注册的 id 都是最大的，全挤在最后一台。你的场景不查范围、只按人独立处理，所以用取模让它均匀。
- **一致性哈希（consistent hashing），先说它解决什么问题**：取模分区有个大麻烦。原来 4 台机器，数据按 id % 4 放。现在加 1 台变 5 台，变成 id % 5。id = 7 原来在 3 号机（7 % 4 = 3），现在要去 2 号机（7 % 5 = 2）。你去算会发现，几乎所有数据都要换机器。加一台机器要把整个库搬一遍，太贵。
- **一致性哈希怎么做**：比方：一条环形跑道，上面标着 0 到 100 的刻度。每台机器站在跑道上某个位置，比如 A 站 10、B 站 40、C 站 70。每条数据算一个哈希值，也是 0 到 100 之间的一个数，落在跑道上。规则只有一条：数据归「顺时针往前走遇到的第一台机器」。哈希值 25 的数据往前走遇到 B，归 B；哈希值 80 的往前走绕过 100 回到 10，遇到 A，归 A。
- **加机器只搬一小段**：新来一台 D 站在 55。原来 41 到 70 之间的数据都归 C，现在 41 到 55 这段归 D，56 到 70 还归 C。只有 C 的一部分数据搬给了 D，A 和 B 的数据一条都没动。有 N 台机器，加一台大约只搬 1/N 的数据，而不是全部。
- **虚拟节点（virtual nodes）**：上面 3 台机器站 10、40、70，看起来还算匀。可要是站成 10、12、14，那 14 到 100 这一大段全归 A，A 累死别人闲着。办法：每台机器不站一个点，站 100 到 200 个点，散在跑道各处。这样每台机器管的总长度就差不多。机器性能强的多站几个点，自然多管一点。
- **热点键（hot key）**：比方：一个超市 100 个收银台，但所有人都挤在卖茅台的那一个。某个 key 的流量特别大（一个大 V 的主页），它落在哪台机器哪台就被压垮。两个办法：① 把这个 key 拆成「key-1、key-2 … key-10」十份，分到十台机器，读的时候十份合起来；② 这个 key 单独给一台专用机器。
- **二级索引（secondary index）**：比方：通讯录按姓名排序，想按电话号码找人就得从头翻到尾。你按 user_id 分区，「查某个人」很快，直接去那一台。但「查所

有在 Google 工作过的人」，每台机器上都可能有，只能每台都问一遍再把结果拼起来（叫 scatter-gather，撒出去再收回来），慢。要想按公司查也快，就另外维护一份按公司名分区的数据（二级索引），写的时候两份都要更新。

你的项目里：取模是对的，因为状态按桶存、同一个人必须回到同一桶，而且桶数你自己定、_meta.json 守着。被问「桶数要变怎么办」：用 history 全量重跑一次换桶数；生产上要平滑扩容就上一致性哈希或者表格式自带的分区。Spark 版 4096 分区；按 user_id 分区让 join 状态表时同一个人在同一台机器上，不用网络搬数据。

I partition by user_id mod 64 because state is stored per bucket and the same user must land in the same bucket next time. Modulo doesn't rebalance gracefully, so for elastic scaling I'd use consistent hashing with virtual nodes, or let the table format own the partitioning.

3 · 复制与一致性

比方：一份账本抄三份放三个地方，一处烧了别处还在。复制 = 同一份数据存多份。

- **主从（leader–follower）**：写只去主，从库跟着抄；读可以分给从库。抄有延迟 → 刚写完去从库读可能读不到（**复制延迟（replication lag）**）。
- **强一致（strong consistency）**：写完所有人立刻看到同样的值。**最终一致（eventual consistency）**：一会儿之后大家才一样。数据流水线的输出通常接受最终一致（按天/按小时更新）。
- **CAP 三个字母**：C = Consistency 一致性，所有机器上读到的值一样。A = Availability 可用性，任何时候来请求都有回应。P = Partition tolerance 分区容忍，网络断成两半时系统还能工作。
- **P 是什么**：比方：两家分店靠电话对账，电话线断了，两家店互相不知道对方卖了什么。这就是「网络分区」，机器都活着，只是彼此联系不上。分布式系统里电话线一定会偶尔断，所以 P 不是你选不选的，是必须扛住的。
- **所以 CAP 实际是二选一**：电话断了的那段时间，分店只能二选一。要么停止卖货等电话恢复（保 C 丢 A：宁可不服务也不让两边账不一样）；要么照常卖、各记各的账，之后再对（保 A 丢 C：一直能用，但这段时间两边读到的值可能不同）。
- **你的场景**：每天一批、输出一次提交，读的人要么看昨天的完整版、要么看今天的完整版，不会读到一半。这是偏 C 的做法，代价是新数据要等到下一次提交才可见，天级新鲜度本来就允许。
- **英文**：P is partition tolerance: the network will split sometimes, so you have to tolerate it. During a split you choose: stay consistent and refuse some requests, or stay available and risk serving stale values. A daily batch with one atomic commit per run is on the consistent side, which the freshness requirement allows.
- **多副本写冲突（write conflict）**：两边同时改同一条 → 要定规则：最后写入赢（last-write-wins）（丢数据）、或者按版本号合并。

你的项目里：输出 parquet 放对象存储（S3 自带多副本），不用自己管复制；每天一版、最终一致。状态表用 Iceberg 的事务提交，读的人永远看到完整的一版，不会读到写一半。

For a daily batch output, eventual consistency is fine: readers see yesterday's complete version until today's commit lands atomically. I'd keep the data in object storage, which already replicates, and use a table format for atomic commits.

4 · 唯一 ID（Snowflake）

比方：车牌 = 省份 + 城市 + 序号，各段各管各的，不用全国排队领号。

- **Snowflake**：64 位 = 1 位符号 + 41 位时间戳（毫秒）+ 10 位机器号 + 12 位每毫秒序号。每台机器自己发号，不用中心协调；按时间大致有序。
- **为什么不用自增 ID（auto-increment）**：要一个中心数据库发号，所有机器都来排队拿号，它成了瓶颈；它挂了谁都发不了号，成了单点。
- **UUID 是什么**：一串随机生成的 128 位数字，长这样 `550e8400-e29b-41d4-a716-446655440000`。好处是谁都能自己生成、几乎不会撞。
- **为什么不用 UUID，三个原因**：
  - **太长**：128 位 = 16 字节，Snowflake 是 64 位 = 8 字节，差一倍。id 会出现在每一行、每个索引、每个外键里，10 亿行就多占 8 GB，而且传输、比较都慢一倍。
  - **无序**：UUID 是随机的，新生成的号可能比旧的小。比方：往一本按编号排好的账本里插条目。编号递增的话，永远往最后一页加，很快。随机编号就要翻到中间某一页插进去，页满了还要把页拆开（数据库叫页分裂）。数据库的主键索引（B 树）就是这本账本，随机 id 让写入慢几倍，索引也变得松散、占更多空间。
  - **不带信息**：Snowflake 的高位是时间戳，看 id 就知道大概什么时候生成的，按 id 排序 = 按时间排序，查「最近一天的记录」直接按 id 范围扫。UUID 看不出任何东西。
- **什么时候 UUID 也行**：数据量小、不做范围查询、或者用新版 UUID v7（前面带时间戳，有序了，只剩「太长」一个缺点）。
- **英文**：A UUID is 128 random bits. It's twice the size of a 64-bit id, which costs space in every row and index, and because it's random, inserts land in random places in the B-tree index and cause page splits, so writes get slower and the index gets fragmented. A Snowflake-style id is half the size and roughly time-ordered, so inserts append at the end and the id itself tells you when it was created.
- **时钟回拨（clock moving backwards）**是 Snowflake 的经典坑：机器时间往回跳会发重号，要等时间追上来再发。

你的项目里：user_id << 16 | 每人序号。同样是「拆段、各管各的」，但不用时间戳，因为要**可复现**：同样输入每次必须发出同样的 id。user_id 最大 32 位 + 16 位序号 = 48 位，64 位放得下；每人上限 65,536，超了报错进隔离，不静默撞号；id 可以因规则 6 合并而退役，退役后永不复用。

Ids are user_id shifted left 16 bits plus a per-user sequence. It's the Snowflake idea, split the bits and let each part be issued independently, but without a timestamp, because the ids must be reproducible from the input.

5 · 缓存

比方：常用的调料放灶台边，不每次去储藏室拿。

- **cache-aside**（最常用）：先查缓存，没有就查库，查到后写进缓存。写数据时删掉缓存里的旧值。
- **TTL**：缓存多久过期。短 = 新鲜但命中低；长 = 命中高但可能旧。
- **淘汰策略（eviction policy）**：满了删谁。LRU 删最久没用的；LFU 删用得最少的。
- **三个坑**：**穿透（cache penetration）**（查不存在的 key，每次打到库 → 缓存空值）；**雪崩（cache avalanche）**（大量 key 同时过期 → TTL 加随机抖动（jitter））。展开：比方，一栋楼所有人的牛奶都订在早上 8 点整送到，8 点电梯就挤爆。缓存也一样：凌晨 0 点一次性把 100 万个 key 写进缓存、都设 1 小时过期，那 1 点整它们同时失效，100 万个请求同一秒全打到数据库，库被压垮。办法：过期时间别设成固定的 1 小时，设成「1 小时 ± 随机几分钟」，比如 55 到 65 分钟之间随机取。这样 100 万个 key 在 10 分钟内零零散散地过期，数据库每秒只多一点点压力。代码就一行：`ttl = 3600 + random.randint(-300, 300)`。Jitter means adding a small random offset to each TTL so keys that were cached together don't all expire in the same second and stampede the database.；**击穿（hot-key breakdown）**（一个热 key 过期瞬间大量请求 → 加锁只让一个去查）。
- **命中率（hit rate）**：命中次数 ÷ 总次数；80% 命中就把库的压力降到 1/5。

你的项目里：lru_cache 缓存 title_key / sim_key / desc_tokens 的结果（同一个头衔反复出现）；TikTok 那段的在线特征链路用过 20 分钟 TTL + 哈希降级，可以拿来当缓存设计的例子。

I'd put a read cache in front of the serving store with cache-aside and a TTL, add jitter to the TTL so keys don't all expire together, and cache negative results so missing keys don't hammer the database.

6 · 消息队列（Kafka）

比方：快递站的传送带。寄件人放上去就走，分拣员按自己的速度拿，带子上的包裹还能倒回去重看。

- **三个用处：解耦（decoupling）**（生产方和消费方不用同时在线）、**削峰（buffering peaks）**（高峰先排队）、**可重放（replayable）**（消费方挂了从上次的位置接着读）。
- **Topic / 分区 / offset**：一个 topic 切成多个分区；同一个分区内有序，跨分区无序；每个消费者记自己读到的位置（offset）。按 key（比如 user_id）分区，同一个人的消息就在同一分区、按顺序到。
  为什么分区内有序、跨分区无序：比方，银行开了 4 个窗口，每个窗口前排一条队。同一条队里，先来的一定先办（这条队有序）。但 1 号窗口的第 5 个人和 3 号窗口的第 2 个人谁先办完，没人管，看各窗口快慢（跨队无序）。Kafka 的每个分区（partition）就是一个窗口：它是一个只往末尾追加的日志（append-only log），每条消息拿一个递增的位置号 offset，消费者按 offset 从小到大读，所以分区内天然有序。不同分区在不同机器上、由不同消费者并行读，没有一个全局的顺序号把它们串起来；要做全局有序，就只能所有消息排同一条队，也就失去了并行（throughput）。所以做法是：需要有序的东西放进同一个分区。用 user_id 当分区键（partition key），同一个人的消息哈希到同一个分区，他自己的事件就保序；不同人之间本来就不需要顺序。Each partition is an append-only log with increasing offsets, so order holds inside it. Partitions are read in parallel with no global sequence across them; global order would mean one partition and no parallelism. So you choose the partition key to match what must stay ordered: user_id keeps each person's events in order.
- **投递语义（delivery semantics）**：至多一次（at-most-once）（可能丢）、至少一次（at-least-once）（可能重）、恰好一次（贵）。常用做法：至少一次 + 消费端幂等。
- **消费者组（consumer group）**：一组消费者分摊一个 topic 的分区，加消费者就加吞吐，但消费者数不能超过分区数。
- **积压（lag）（consumer lag）**：生产比消费快，offset 落后越来越多，是最重要的监控指标。

你的项目里：每天的新观测可以从 Kafka 进来，按 user_id 分区，这样迟到数据和正常数据在同一个人的分区里；消费端用你的 step 函数，它本来就是幂等可重放的（同样的输入同样的输出）。

New observations would arrive on a Kafka topic partitioned by user_id, so one person's events stay ordered. Delivery is at-least-once and the consumer is idempotent, because step is deterministic, so a redelivered message can't produce a different result.

7 · 批处理 vs 流处理，迟到数据

比方：批处理是每天晚上结一次账；流处理是每笔交易当场记。

- **批**（Spark、MapReduce）：吞吐大、简单、延迟小时到天。**流**（Flink、Kafka Streams）：秒级延迟，但要处理乱序、迟到、状态管理，运维重。
- **事件时间 vs 处理时间（event time vs processing time）**：事情发生的时间 vs 系统看到的时间。两者差 = 迟到。**水位线（watermark）**：系统宣布「X 时刻之前的事件应该都到了」，之后再来的算迟到，要单独处理（丢、或者回头更正）。
- **Lambda 架构**：批 + 流两套，流给实时近似，批晚上修正。**Kappa 架构**：只用流，要重算就从队列头重放。
- **MapReduce 核心**：map 按 key 打散 → shuffle 把同 key 聚到一台 → reduce 处理。贵的是 shuffle（网络搬数据）。
- **数据倾斜（data skew）**：某个 key 的数据特别多，一台机器干到天亮别的都闲着。办法：热 key 加盐拆开（salting）、两阶段聚合（two-phase aggregation）、或者单独处理大 key。

你的项目里：选批，因为题目只要每天更新，流的复杂度用不上。observed_ts 是事件时间、到达日期是处理时间，迟到 = 两者不一致，你的处理是「只重放那一个人」，相当于迟到数据的回头更正。Spark 的 repartition by user_id 就是 shuffle；人均 29 条、最多 166 条，没有倾斜；真有超级用户就单独处理。

The brief needs daily freshness, so batch is the right tool; streaming would add ordering and state complexity for no gain. observed_ts is event time, arrival is processing time, and late data is handled by replaying just that person, which is the batch version of a watermark correction.

8 · 幂等、恰好一次、确定性重算

比方：电梯按钮按十次和按一次一样。幂等 = 同一个操作做多次（idempotent），结果和做一次一样。

- **为什么重要**：分布式里重试不可避免（网络超时不知道成没成功），只有幂等才能放心重试。
- **怎么做到**：操作带唯一 id，做过就跳过（去重表（dedup table））；或者操作本身是「设置成 X」而不是「加 1」。
- **恰好一次（exactly-once）**通常 = 至少一次 + 幂等，或者事务性写入（输出和 offset 一起提交）。
- **确定性（deterministic）**：同样输入同样输出，没有随机、没有依赖时间或机器。有了确定性，重跑就是最简单的修复手段。

你的项目里：id 从 user_id 和首次出现顺序算出来，没有全局计数器 → 确定性；测试「跑两次结果一样」「全量 = 半量 + 增量」就是幂等和确定性的证明；输出文件先写临时再改名（原子写），跑一半挂了不会留半个文件；迟到数据直接重放，因为重放是安全的。

The pipeline is deterministic: same input, same ids, same output, with no global counter and no randomness. That's what makes retries and replays safe, and it's covered by tests: run twice equals once, and full equals half plus incremental.

9 · 存储：对象存储、列式文件、数据湖表格式

比方：S3 是无限大的仓库，parquet 是按列装箱的货物，Iceberg 是仓库的账本（哪些箱子算当前这一版）。

- **对象存储（S3）（object storage）**：便宜、无限、自带多副本；但没有「追加」和「改一行」，只能整文件写。延迟高（几十毫秒），适合批不适合点查。
- **列式（parquet）（columnar）**：同一列存在一起。好处：只读需要的列（你 conform 只读 10 列）；同列压缩率高；带统计信息（每块的最大最小值）能跳过整块（**谓词下推（predicate pushdown）**）。坏处：改一行要重写文件。
- **行式（row-oriented）**（数据库表）：一行存一起，点查快、改一行快。
- **表格式（Iceberg / Delta / Hudi）（table format）**：在一堆 parquet 文件上加一层元数据，提供：**ACID 提交（ACID commit）**（一次写入要么全可见要么全不可见）、**快照 / 时间旅行（snapshot / time travel）**（回到任何一版）、**MERGE**（按键更新）、**分区裁剪（partition pruning）**（按分区只读需要的文件）、**schema 演进（schema evolution）**（加列不重写）、**小文件合并（compaction）**。
- **Iceberg 的「事务提交」具体是怎么做到的**：
  - 比方：仓库里放着一堆箱子（parquet 文件），墙上有一块公告板，贴着一张清单：「当前这一版 = 这 500 个箱子」。读的人只认清单，清单上没有的箱子当不存在。
  - 写新数据的流程：① 先把新箱子搬进仓库（写新的 parquet 文件，旧文件一个都不改）；② 写一张新清单，列出新版本包含哪些箱子（旧的 480 个 + 新的 30 个，去掉被替换的 20 个）；③ 最后一步，把公告板上的清单换成新的。这一换是一个原子动作，要么换了要么没换，没有中间状态。
  - 所以：写到一半挂了，新箱子搬了一半但清单没换，读的人还是看旧清单，什么都没变，只是仓库里多了几个没人认领的箱子，事后清理。这就是「要么全可见要么全不可见」。
  - 读的人在你换清单的那一刻之前拿到的是旧清单，之后拿到的是新清单，永远不会拿到一张写了一半的清单。读写互不阻塞。
  - 旧清单不扔，所以可以「回到上周三那一版」（时间旅行），也可以一键回退。
  - 两个人同时写：换清单时要先检查「公告板上现在还是我开始时看到的那张吗」。是，就换；不是（别人先换了），就拿新清单重做一遍再换。这叫乐观并

发（optimistic concurrency），和 git push 被拒绝要先 pull 一样。
    - 术语对照：清单 = 元数据文件（metadata.json → snapshot → manifest），公告板 = catalog（Hive Metastore / Glue / Nessie），换清单 = 对 catalog 做一次 compare-and-swap。

  Iceberg never modifies a data file. A write adds new parquet files, writes a new metadata snapshot listing exactly which files make up the new version, and then atomically swaps the table's pointer in the catalog. Readers only follow the pointer, so they see either the old version or the new one, never a half-written state. Old snapshots stay, which gives time travel and rollback, and concurrent writers use compare-and-swap on the pointer.

- **小文件问题（small-files problem）**：几百万个 1 MB 文件比几千个 1 GB 文件慢得多（打开文件有固定开销）。目标每个文件 128 MB–1 GB。

你的项目里：输出 parquet、每桶一个文件（64 个），大小合适；状态现在是 parquet 每天整份重写，生产应放 Iceberg 按 user_id 分区、MERGE 只更新动过的人、一次运行一次提交、旧版本留着回退；history 只追加。桶太多 = 小文件问题，所以桶数不是越多越好。

Output goes to parquet on object storage, one file per bucket. For state I'd use Iceberg partitioned by user_id: merge only touched users, one atomic commit per run, and snapshots so a bad run rolls back. Parquet's column pruning and min-max statistics are why reading only ten columns is cheap.

10 · 实体消歧 / 去重系统（record linkage）

比方：两本通讯录合并，「张伟 13800」和「Zhang Wei 138-00」是不是同一个人。

- **三步**：**Blocking（blocking / candidate generation）**（先用便宜的键把候选缩小，比如同雇主）→ **成对比较（pairwise comparison）**（打分：相似度 + 规则或模型）→ **聚类 / 传递闭包（clustering / transitive closure）**（A=B、B=C → A=C，合成一簇）。
- **Blocking 为什么必须**：N 条两两比是 N^2；1000 亿行不可能。按 user_id 和雇主分块后，每人只有几十条。
- **规则 vs 模型**：规则可解释、可复现、快；模型（逻辑回归 / 梯度提升 / embedding）召回高但要标注、要可复现地部署。常见做法：规则做主、模型只用在规则拿不准的那层。
- **评估**：成对指标（pairwise metrics）（配对的精确率 / 召回）vs 簇指标（B-cubed）（cluster metrics, B-cubed）；没有标准答案时抽样 + 判官 + 人工 + 无标注健康度。
- **传递性的坑（chaining problem）**：A 像 B、B 像 C，但 A 和 C 一点不像，链式合并会把不相关的合成一团。要么限制簇大小，要么要求和簇里多数相似。

你的项目里：规则 2 就是 blocking（同雇主 + 日期兼容才算候选）；规则 4–5 是成对比较；规则 6 是把已发出的 id 做传递合并，但只在被动过的职位上做、且要同开始日期，避免链式乱合。学习打分器试过、没上线，下一步只用在弱点层。

It's a standard record-linkage shape: blocking by employer and date to keep candidates small, pairwise scoring with rules, then a constrained merge of duplicate ids. Rules first because they're reproducible and explainable; a learned scorer only where the rules are unsure.

11 · 数据质量监控与可观测性

比方：工厂流水线上的质检员，不等客户投诉才发现次品。

- **三类指标（volume · quality · freshness）**：**量**（行数、人数、文件数，和昨天比差多少）、**质**（空值率、重复率、日期倒序数、覆盖率）、**时效**（数据几点到、跑了多久）。
- **告警规则**：和历史同期比，偏离 3 个标准差或者某个百分比就报警；绝不能为 0 的指标（主键重复、倒序日期）直接硬断言。
- **数据契约（data contract）**：上游保证 schema 和含义不变，变了要通知；你的 conform 「缺列直接报错」就是契约的硬执行。
- **金丝雀 / 回归测试（canary / regression test）**：固定一小份输入，每次跑完对比输出是否变化。
- **隔离而不是整体失败（quarantine, partial failure）**：坏的一小部分进隔离区，其余照常出，事后看隔离区。
- **血缘（lineage）**：这列数据从哪来、经过哪些步骤，出了问题能追。

你的项目里：run.json 里的指标（输入行、输出行、丢弃行按规则、修复格数、合并数、重放人数、隔离人数）就是质量看板；覆盖率 99.94%、倒序日期 0、可疑拆分 3.7 万是无标注健康度；quarantine 目录是隔离区；51 个测试里的金丝雀。被问「上线后怎么知道结果还好」：每天看这几个数的走势，突变就查。

Every run emits metrics: rows in and out, drops per rule, repairs, merges, replays, quarantined users. I'd chart those daily and alert on deviations, hard-fail on anything that must be zero like duplicate keys or inverted dates, and keep the quarantine folder so one bad user never blocks the run.

12 · 服务层：把结果给下游用

比方：仓库里的货要上货架，顾客才拿得到。批处理的结果要有人能查。

- **两种读法**：**分析查询（analytical query）**（整表扫描、聚合）→ 直接读 parquet / 数据仓库（BigQuery、Snowflake、Trino）；**点查（point lookup）**（给我这个人的履历）→ 要 KV 存储（DynamoDB、Cassandra、Redis），按 user_id 做键，毫秒级。
- **批到线上的搬运**：每天跑完把结果批量写进 KV（bulk load），或者只写动过的人。
- **API 设计**：GET /users/{id}/positions?asof=2026-10-01；分页；版本号；限流（rate limiting）。
- **Cassandra / DynamoDB 一句话**：按分区键哈希分片、多副本、最终一致、写快读快，不擅长复杂查询和 join。
- **读写分离（read/write separation）**：批处理写一份，线上读另一份，互不影响。

你的项目里：下游要查「某人截至某天的履历」→ 按 (user_id, asof_ts) 做键放 KV；每天只有动过的人要更新，正好和增量输出对上；restated 名单告诉下游哪些人的历史行要整体替换。

For point lookups I'd load the daily output into a key-value store keyed by user and as-of date, updating only the users touched that day, and the restated list tells the loader which users need their history replaced.

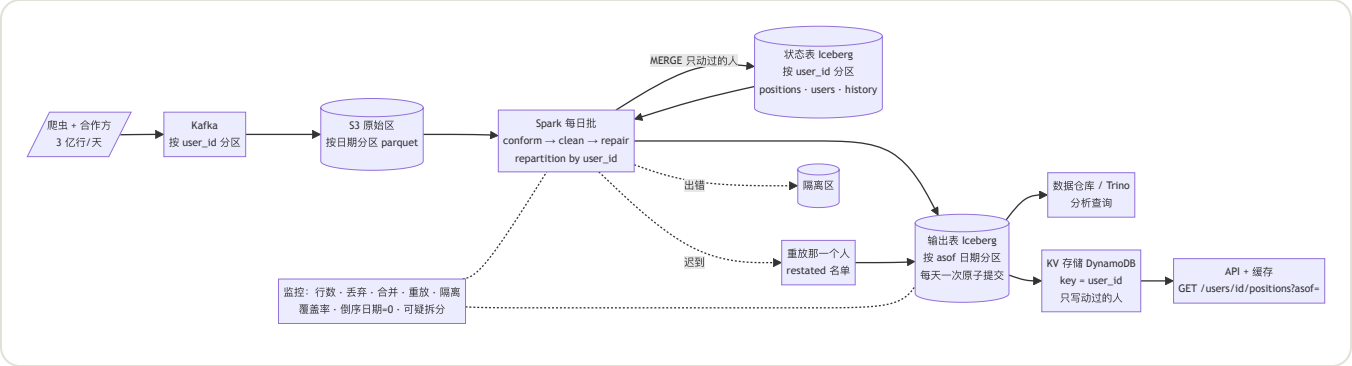
押题最终答案（三道，每道一张图 + 最优设计）

补充手册第 2 章有思路，这里给的是「最后要落到的那张图」。答不出来就画图、按框念。

题 A · 每天几亿条档案进来，维护去重后的「每人职业经历」（= 你的 take-home 上生产）

先问：每天多少条（设 3 亿）、新鲜度（天级够）、下游谁读（分析 + 点查）、要不要保留历史版本（要，as-of）。

估算：3 亿行/天 × 500 B = 150 GB/天；存量 1000 亿行 ≈ 50 TB；每天只有约 1% 的人有新数据；按 3.5 万行/核秒，每天增量 3 亿行 ≈ 2.4 核时，几台机器半小时。



- **核心**：和作业一样的 coalesce_bucket，Spark mapInArrow 调同一个函数；按 user_id 分区所以状态 join 不用跨机器。
- **状态**：Iceberg 按 user_id 分区、只 MERGE 动过的人、一次运行一次提交、旧快照留着回退；history 只追加、只给迟到的人读。
- **迟到**：事件时间 observed_ts；比状态旧的只重放那一个人，restated 告诉 KV 加载器整体替换。
- **隔离**：一个人出错回滚进隔离区，不阻塞当天运行。
- **服务**：分析走仓库；点查走 KV（key = user_id，value = 当前履历 + as-of 索引）+ 读缓存。
- **扩容**：人独立 → 加 executor 线性；分区数调到每分区几十万行；超级用户单独处理。
- **取舍**：天级批而不是流（新鲜度要求天级，流的乱序和状态管理不值）；规则而不是 LLM 逐对（可复现、可解释、1000 亿行成本）；merge when in doubt。
- **隐私（right to be forgotten）**：删除名单在输出时过滤。

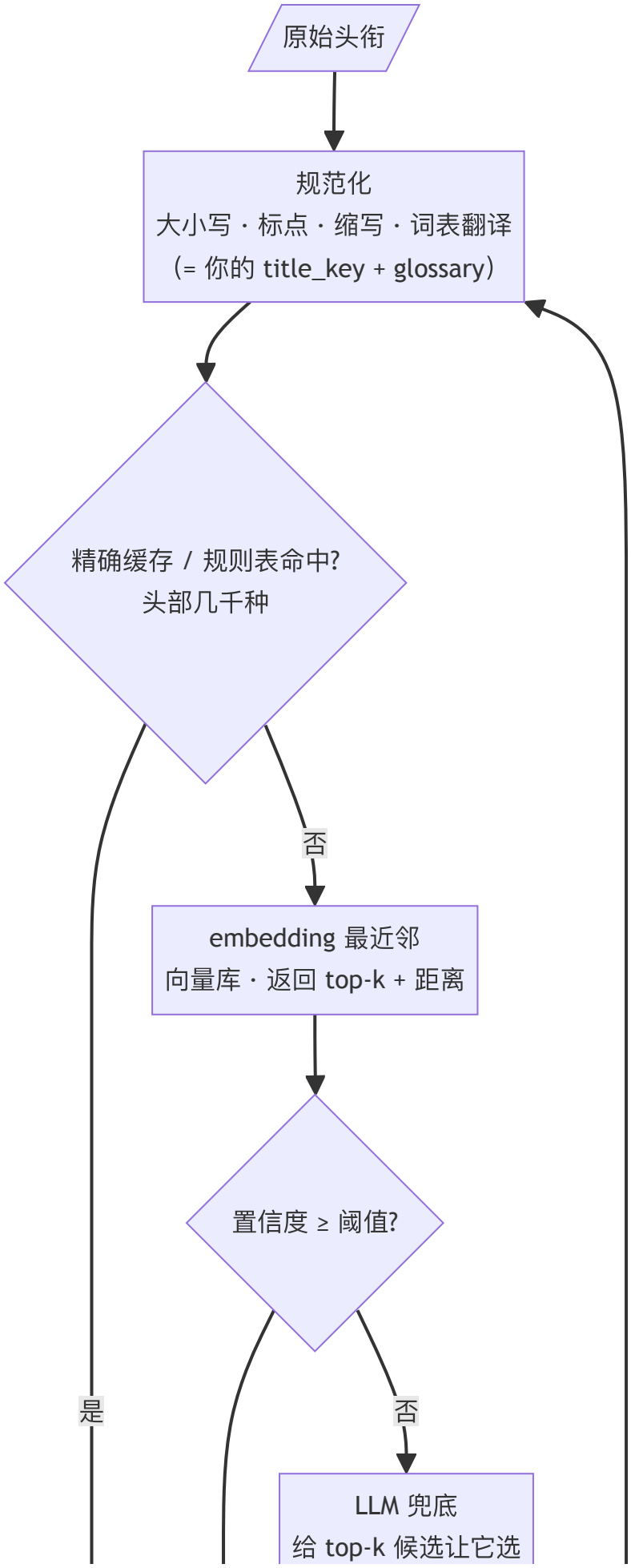
Same core as the take-home, run under Spark partitioned by user_id, with state in an Iceberg table partitioned by user_id so each day merges only touched users and commits atomically. Late data replays one person; failures are quarantined. Output

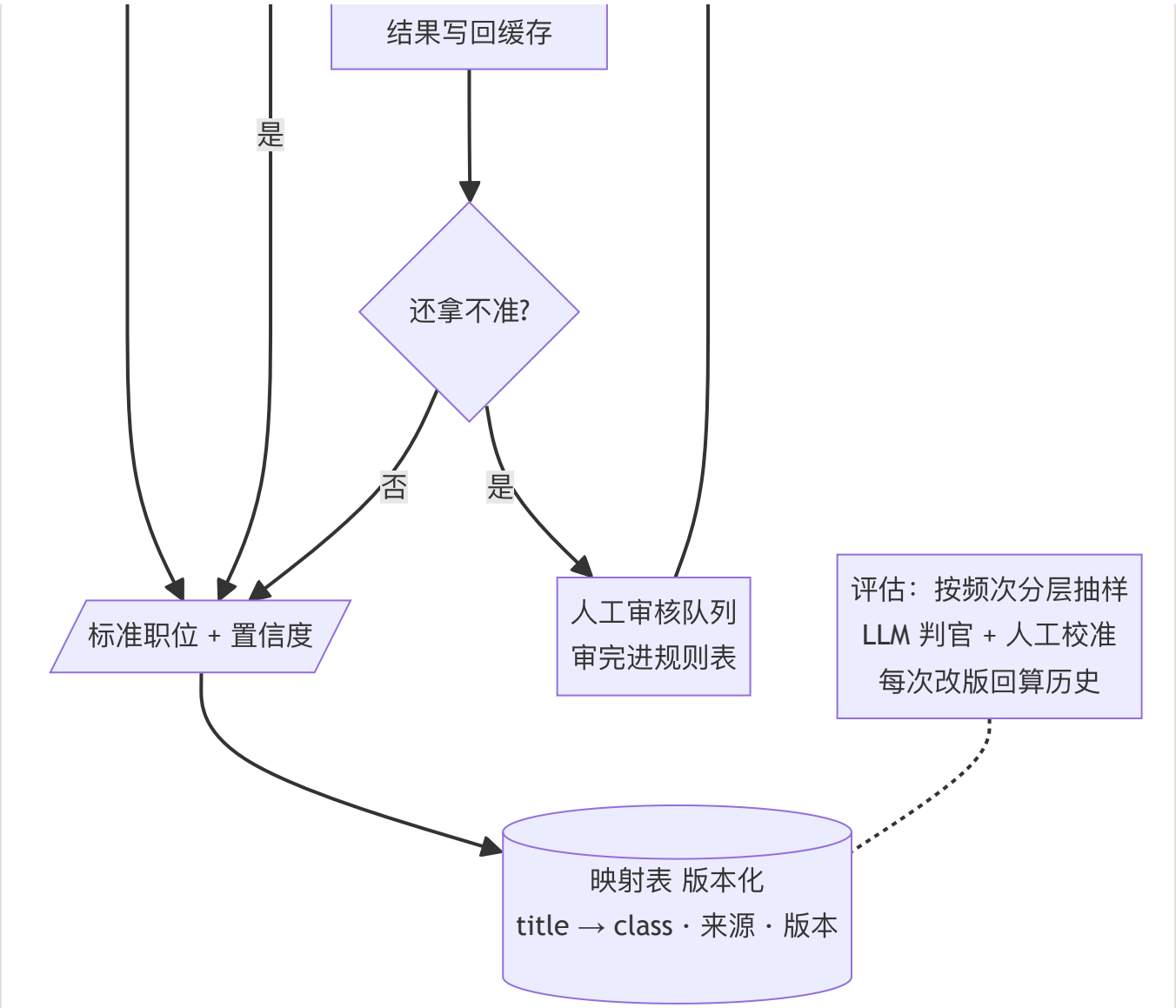
goes to a warehouse for analysis and a key–value store for point lookups, and every run emits the quality metrics I alert on.

题 B · 职位名 / 技能映射服务（1000 万种原始职位名 → 标准分类）

先问：分类表多大（几千类）、延迟要求（线上毫秒 vs 离线批）、准确率目标、要不要多语言、分类表会不会改版。

估算：去重后约 1000 万种头衔，但头部几千种覆盖 80% 流量；线上 QPS 设 1 万；每个向量 384 维 × 4 B ≈ 1.5 KB，1000 万 × 1.5 KB = 15 GB，一台机器放得下。


原始头衔
规范化
大小写 · 标点 · 缩写 · 词表翻译
(= 你的 title_key + glossary)
精确缓存 / 规则表命中?
头部几千种
否
embedding 最近邻
向量库 · 返回 top-k + 距离
置信度 ≥ 阈值?
是
否
LLM 兜底
给 top-k 候选让它选



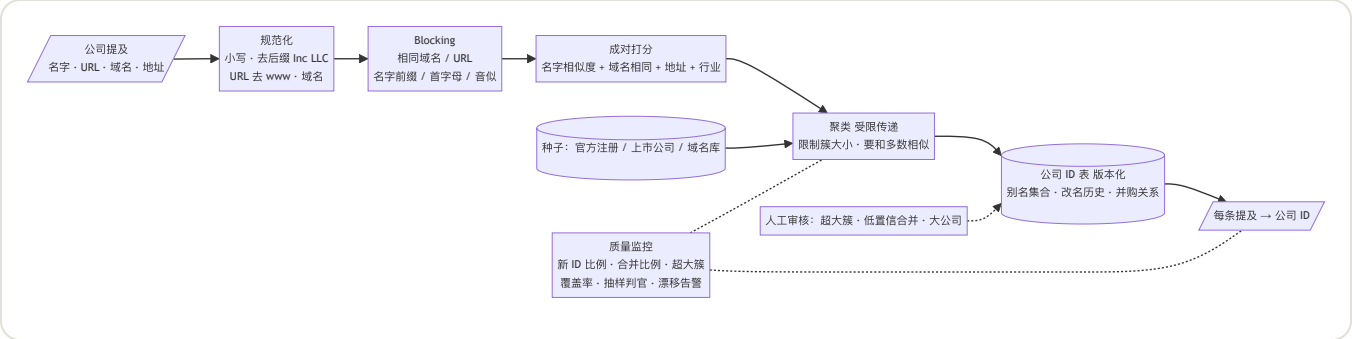
- **三层漏斗**：规则/缓存吃掉大头（毫秒、确定）；embedding 吃长尾（几毫秒）；LLM 只处理低置信（贵、慢、结果缓存后不再调）。
- **为什么不全用 LLM**：1000 万次调用成本，答案不稳定，线上延迟；LLM 的价值是兜底 + 当判官。
- **版本化**：分类表改版要能回算历史，不然时间序列断；映射表记来源（规则 / 向量 / LLM / 人工）。
- **评估**：按频次分层抽样（头部错一个影响大）；LLM 判官用人工样本校准；监控「进 LLM 的比例」「进人工的比例」，涨了说明有新词。
- **多语言**：多语言 embedding 模型；词表先翻译。
- **取舍**：准确 vs 延迟 vs 成本，用置信度阈值调；阈值高 → 更多进 LLM/人工。

A three-tier funnel: normalise, then rules and a cache for the head of the distribution, embedding nearest-neighbour for the long tail, and an LLM only for low-confidence cases, with results cached and the lowest sent to human review. The mapping table is versioned so history can be recomputed, and I evaluate on frequency-stratified samples with an LLM judge calibrated against human labels.

题 C · 公司实体消歧 + 数据质量监控

先问：公司实体来源（档案里的公司名、URL、官网、注册信息）、要合并到什么粒度（法人 vs 品牌 vs 子公司）、改名/并购怎么算、下游要什么（公司级统计）。

估算：几亿条公司提及，去重后几千万个名字，真实公司几百万；两两比不可能 → 必须 blocking。



- **三步**：blocking（域名最强、名字前缀 / 首字母 / 音似兜底）→ 成对打分（名字 + 域名 + 地址 + 行业）→ 受限聚类（限制簇大小，防 A 像 B、B 像 C 的链式乱合）。
- **别名集合 + 改名历史**：一个公司记所有叫过的名字和域名；并购是关系不是合并（Jeppesen/Boeing 那种）。
- **种子数据**：官方注册库、上市公司、域名库当锚点，先把大公司钉死。
- **质量监控**：每天看新 ID 比例、合并比例、最大簇大小、覆盖率；分层抽样 + 判官；突变告警（抓取器变了会先体现在这里）。
- **人工**：只审三类：超大簇、低置信合并、头部大公司。
- **版本化**：ID 永不复用、可退役；历史可回算。
- **取舍**：合并错（两家公司算一家，统计错）比漏合并（一家算两家）更伤 → 这里和作业相反，存疑就分开，靠人工和种子来合。

Standard record linkage: normalise names and domains, block on domain and name prefix, score pairs on name similarity plus domain, address and industry, then cluster with a cap on cluster size to avoid chaining. Each company keeps an alias set and rename history; mergers are relationships, not merges. Official registries seed the big companies. For quality I track new-id and merge rates, max cluster size and coverage daily, sample with judges, and route oversized or low-confidence clusters to human

review. Here a wrong merge hurts more than a missed one, so unlike the take-home I split when in doubt.

13 · 其他常被问的一页

- **负载均衡（load balancer）**：把请求分给多台机器（轮询 / 最少连接 / 按哈希）；本身要双活防单点。
- **限流**：令牌桶（token bucket）（匀速放令牌，有令牌才能过，允许短突发）；滑动窗口计数。
- **CDN**：静态内容放到离用户近的节点。
- **数据库索引（B-tree index）**：B 树，像书的目录；加速读、拖慢写；查什么就按什么建。
- **SQL vs NoSQL**：要事务和复杂查询用 SQL；要水平扩展、简单键值、灵活 schema 用 NoSQL。
- **分布式锁 / 单写者（distributed lock / single writer）**：同一时刻只允许一个人改，租约 + 过期防止持锁者挂掉。
- **隐私**：被遗忘权 → 删除名单在输出时过滤；日志不打描述原文。
- **容错（fault tolerance: retry, backoff, circuit breaker）**：重试 + 退避 + 幂等；超时；断路器（下游挂了就先别打）；单桶重试而不是整体重跑。

14 · 你的项目 ↔ 概念 速查

你做了的	概念名（答题时点名）
user_id % 64 分桶、同人同桶	按键分区 / 分片；取模扩容要搬家 → 一致性哈希
每人状态机、按日期一次一个快照	有状态处理、事件时间、不用未来信息
指纹跳过 57.5%	内容哈希去重
user_id << 16 | 序号	Snowflake 式分段 ID、可复现
规则 2 找候选 → 规则 5 打分 → 规则 6 折叠	Blocking → 成对比较 → 受限的传递合并（record linkage）
state-in / state-out、只处理有新数据的人	增量计算、检查点
迟到数据只重放一个人	事件时间 vs 处理时间、水位线后的更正
全量 = 半量 + 增量、跑两次一样	幂等、确定性
一个人出错回滚进隔离	故障隔离、部分失败不阻塞
先写临时文件再改名	原子写
cgroup CPU 配额、每进程 1 线程	容器资源感知、避免线程超订
Spark mapInArrow 跑同一函数	MapReduce / shuffle by key、Arrow 零拷贝
状态放 Iceberg 按 user_id 分区	数据湖表格式：ACID、MERGE、分区裁剪、时间旅行
run.json 指标、覆盖率、倒序日期 0	数据质量监控、无标注健康度
判官 + 人工 + 回归测试	离线评估、金丝雀

15 · 英文口述版 · 一句话库

- Partitioning: "I'd partition by user_id so all of one person's data lands on one worker; nothing is shared across people, so it scales linearly."
- Consistency: "It's a daily batch, so eventual consistency is fine as long as each day's output is published atomically."
- Queue: "A Kafka topic partitioned by user_id keeps one person's events in order; the consumer is idempotent so at-least-once delivery is safe."
- Batch vs stream: "The freshness requirement is daily, so batch wins; streaming would add ordering and state complexity with no benefit."
- Late data: "observed_ts is event time; when something arrives late I replay only that user, which is cheap and verifiable."
- Storage: "Parquet on object storage for the output; Iceberg for the state so I can merge only touched users and roll back a bad run."
- Monitoring: "Each run emits counts per rule; I'd alert on deviations and hard-fail on anything that must be zero."
- Trade-off line: "I chose X over Y because of Z; the cost is W, and here's how I'd notice if W becomes a problem."